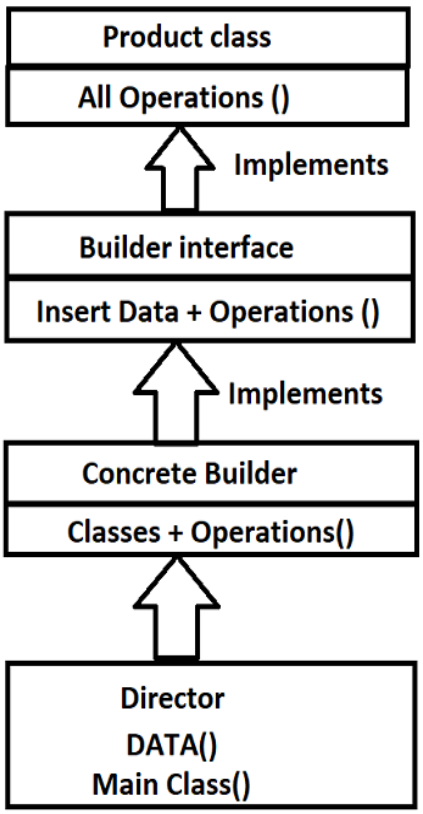
Key components of the Builder Pattern in C++

- **Director:** The Director is the main component of the builder pattern it basically responsible for the construction process of the program. It works with a Builder to build an object. The Director knows the actual steps required to build an object, but it does not know the details of how each step is implemented.

- **Builder:** The Builder is the main interface or an abstract class that defines the construction steps required to create an object.

- **Concrete Builder:** Basically, these are the classes that implement the Builder interface. Each Concrete Builder is responsible for constructing a particular variant of the object.

- **Product:** The Product is the complex object that we want to create. The Product class may have methods to access or manipulate these components. It often has multiple parts or components that are built by the Builder.



```
#include <iostream>
#include <string>

// Product class
class Pizza {
public:
	void setDough(const std::string& dough)
	{
		this->dough = dough;
	}

	void setSauce(const std::string& sauce)
	{
		this->sauce = sauce;
	}

	void setTopping(const std::string& topping)
	{
		this->topping = topping;
	}

	void displayPizza() const
	{
		std::cout << "Pizza with Dough: " << dough
				<< ", Sauce: " << sauce
				<< ", Topping: " << topping << std::endl;
	}

private:
	std::string dough;
	std::string sauce;
	std::string topping;
};

// Abstract builder class
class PizzaBuilder {
public:
	virtual void buildDough() = 0;
	virtual void buildSauce() = 0;
	virtual void buildTopping() = 0;
	virtual Pizza getPizza() const = 0;
};
```

```
// Concrete builder for a specific type of pizza
class HawaiianPizzaBuilder : public PizzaBuilder {
public:
	void buildDough() override
	{
		pizza.setDough("Pan Dough");
	}

	void buildSauce() override
	{
		pizza.setSauce("Hawaiian Sauce");
	}

	void buildTopping() override
	{
		pizza.setTopping("Ham and Pineapple");
	}

	Pizza getPizza() const override { return pizza; }

private:
	Pizza pizza;
};

// Concrete builder for another type of pizza
class SpicyPizzaBuilder : public PizzaBuilder {
public:
	void buildDough() override
	{
		pizza.setDough("Thin Dough");
	}

	void buildSauce() override
	{
		pizza.setSauce("Spicy Tomato Sauce");
	}

	void buildTopping() override
	{
		pizza.setTopping("Pepperoni and Jalapenos");
	}

	Pizza getPizza() const override { return pizza; }

private:
	Pizza pizza;
};
```

```
// Director class that orchestrates the construction
class Cook {
public:
	void makePizza(PizzaBuilder& builder)
	{
		builder.buildDough();
		builder.buildSauce();
		builder.buildTopping();
	}
};

int main()
{
	Cook cook;

	HawaiianPizzaBuilder hawaiianBuilder;
	cook.makePizza(hawaiianBuilder);
	Pizza hawaiianPizza = hawaiianBuilder.getPizza();
	hawaiianPizza.displayPizza();

	SpicyPizzaBuilder spicyBuilder;
	cook.makePizza(spicyBuilder);
	Pizza spicyPizza = spicyBuilder.getPizza();
	spicyPizza.displayPizza();

	return 0;
}
```